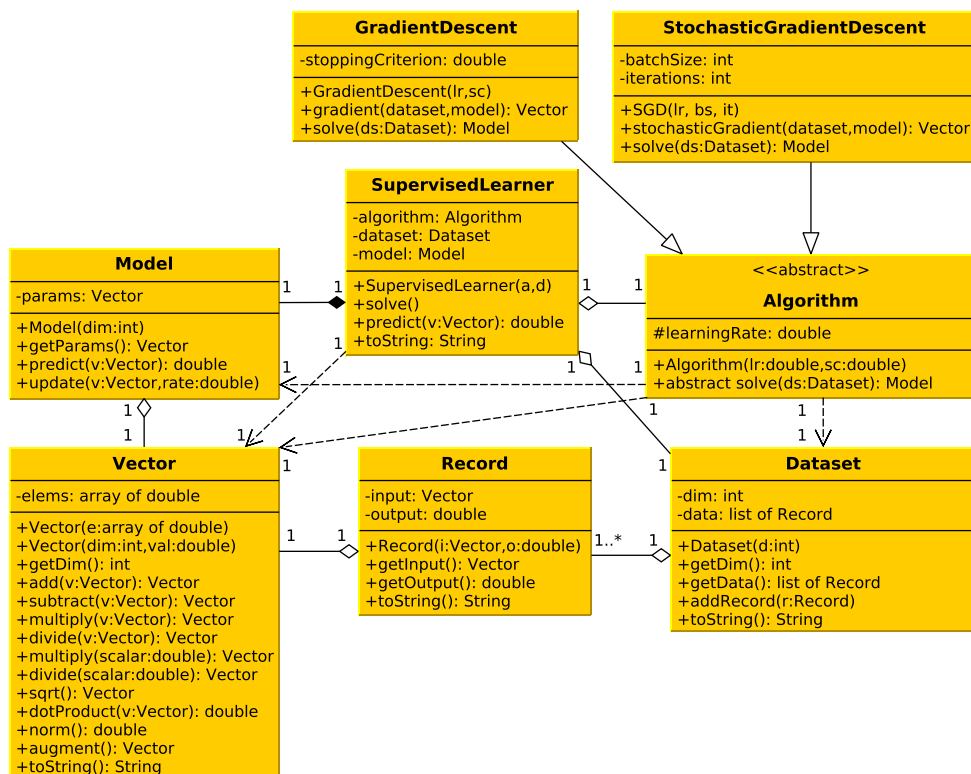


Lab 5: Inheritance in C++
24292-Object Oriented Programming

1 Introduction

In this lab the aim is to implement linear regression in C++. A simplified class diagram without standardized datasets appears below. The classes Vector, Record, Dataset and Model have already been implemented and are provided in the supplied code. Your task is to implement the remaining four classes (Algorithm, GradientDescent, StochasticGradientDescent and SupervisedLearner) and update the main function in TestLearner.cpp to test the classes.



Please have a look at the code of the supplied classes in order to understand how they work. Note that constructors often initialize the values of attributes before the method body starts. Visibility is applied in entire blocks of private and public, not individually for each attribute or method. We can create instances without using the `new` keyword by ending the declaration of an instance with parentheses. The following example creates a vector of dimension 5 whose elements are initialized to 1, equivalent to calling the first constructor of `Vector`:

```
Vector vec(5, 1);
```

It is possible to use the `new` keyword, but this creates a *pointer*, and the memory has to be manually deleted using the `delete` keyword. Hence we recommend creating class instances using the above syntax in the lab.

Instead of implementing a method that returns a string, in C++ we can directly implement a method for passing an instance of a class to an output stream using the operator `<<`. (The `friend` modifier indicates that the operator is *external* to the class but is allowed access to the private attributes of the class.) The classes `Vector` and `Dataset` use the existing `std::vector` class of C++ to store collections of elements. Unlike Java, class definitions end with a semicolon `;`).

1.1 Include directives

Unlike Java, other header files in the same directory are not included by default, so it is necessary to explicitly include the header files that we want to use. Inclusion is recursive, so if `TestLearner.cpp` includes `Dataset.h` and `Dataset.h` includes `Record.h`, then `TestLearner.cpp` recursively includes `Record.h` (and `Vector.h`) as well. The header `bits/stdc++.h` included in `Vector.h` is sufficient to include all built-in classes and functions in C++ that you will need.

If two or more C++ files include the same header file (e.g. `Record.h` and `Model.h` both include `Vector.h`), then by default all definitions in the header file are *duplicated*, which usually causes compiler errors. To avoid duplication, C++ header files typically define *include directives* as follows:

```
#ifndef __VECTOR__
#define __VECTOR__

// all code of the header file goes here

#endif
```

The first time a header file is included, all definitions become active, but the second time, `__VECTOR__` is already defined, preventing duplication of definitions. If you want, try removing the include directives from `Vector.h` and see what happens when you compile.

2 Algorithms

The `Algorithm` class should have a protected attribute so that the subclasses can access it. The explicit modifier `abstract` does not exist in C++, so the class header is the same as for concrete classes. To implement the abstract method `solve`, the method declaration should simply end with “= 0;” without brackets. This implicitly makes the class abstract such that it is not possible to create instances. To enable subclasses to override a method, the method declaration has to include the `virtual` modifier.

The `GradientDescent` and `StochasticGradientDescent` classes should both inherit from `Algorithm` and override the method `solve`. In C++ inheritance is indicated in the class header as follows:

```
class GradientDescent : public Algorithm
```

The keyword `super` does not exist in C++, and to call the constructor of the superclass we instead use the *name* of the superclass.

The remaining code for `GradientDescent` and `StochasticGradientDescent` can mostly be copied from the Java code developed in Lab 4 with few modifications. However, to sample a batch of records from a dataset, the C++ code is different:

```
std::vector<Record> batch;  
std::sample(data.begin(), data.end(), std::back_inserter(batch),  
            batchSize, std::mt19937{ std::random_device{}() } );
```

The collection `batch` now contains the subset of records that belong to the batch.

3 The SupervisedLearner class

The `SupervisedLearner` class is mostly identical to the Java class from Lab 4, with the appropriate changes for C++: include directives, attribute initialization before the brackets, visibility in blocks, and a definition of the operator << for printing. Since there are no standardized datasets, the method `solve` does not need to transform the input or output, but can directly augment the vector given as argument and return the output of the method `predict` of `Model`.

4 Testing

Implement code in the main method of `TestLearner.cpp` to test that the code works as intended. Using the same hyperparameters as in Lab 4 should give good results here as well.

5 Submission

The source code and documentation should all be uploaded to a directory **Lab5** of your Git repository **before 21 December**. Please make sure that the directory name is *exactly* **Lab5** (not **Lab_5**, **Lab-5**, **LAB5**, **lab5** etc.), and that **Lab5** is located in the **root of the Git repository**. Failure to comply with these rules will result in a grade deduction.